

Artificial Intelligence (AI2002)

Assignment 01 Report

Pacman Search Algorithms

- Eman Hassan | Roll Number: 24i3082
- Aun Ulfat | Roll Number: 24i 3068
- Salman Ahmed | Roll Number: 24i 3004

Table of Contents

Table of Contents.....	1
0. Preliminary Lemma : Geometric Node Count Bound.....	1
Lemma.....	1
Proof.....	1
1. Task 1 : Depth First Search (DFS).....	1
1.1 Goal and Approach.....	1
1.2 Algorithm Steps.....	1
1.3 Time Complexity Derivation.....	1
1.4 Space Complexity Derivation.....	1
1.5 Completeness and Optimality.....	1
1.6 Test Commands and Screenshots.....	1
2. Task 2 : Breadth First Search (BFS).....	1
2.1 Goal and Approach.....	1
2.2 Time Complexity Derivation.....	1
2.3 Space Complexity Derivation.....	1
2.4 Completeness and Optimality.....	1
2.5 Test Commands and Screenshots.....	1
3. Task 3 : Uniform Cost Search (UCS).....	1
3.1 Goal and Approach.....	1
3.2 Time Complexity Derivation.....	1
3.3 Space Complexity Derivation.....	1
3.4 Completeness and Optimality.....	1
3.5 Test Commands and Screenshots.....	1
4. Task 4 : Greedy Best First Search (GBFS).....	1
4.1 Goal and Approach.....	1
4.2 Time Complexity Derivation.....	1
4.3 Space Complexity Derivation.....	1
4.4 Completeness and Optimality.....	1
4.5 Test Commands and Screenshots.....	1
5. Task 5 : A Star Search.....	1
5.1 Goal and Approach.....	1
5.2 Admissibility and Consistency, Defined.....	1
5.3 Time and Space Complexity Derivation.....	1
5.4 Formal Proof That A Star Is Optimal With An Admissible Heuristic.....	1
5.5 Test Commands and Screenshots.....	1

6. Task 6 : Multi Goal State Space (Corners Problem)	1
6.1 Goal and Approach.....	1
6.2 cornersHeuristic Design and Admissibility Proof.....	1
6.3 Test Commands and Screenshots.....	1
7. Task 7 : Eating All Food Dots and Nearest Food Search	1
7.1 foodHeuristic for FoodSearchProblem, and Its Admissibility.....	1
7.2 AnyFoodSearchProblem.....	1
7.3 Test Commands and Screenshots.....	1
8. Automated CSV Trace Logging	1
9. Custom Maze Creation and Experiments	1
9.1 Layout Design.....	1
9.2 Running All Five Algorithms on the Custom Layout.....	1
9.3 Empirical Comparison.....	1
10. Complexity Summary Table	1

0. Preliminary Lemma : Geometric Node Count Bound

This lemma is proved once here and then applied directly inside the complexity derivations for DFS, BFS, GBFS, UCS and A Star, instead of repeating the same algebra five times.

Lemma

Let b be a branching factor with b greater than 1, and let k be a non negative integer. Suppose a search process generates at most b to the power i nodes at depth i , for every i from 0 to k . Let $N(k)$ denote the total number of nodes generated from depth 0 up to and including depth k . Then:

$$N(k) = (b^{(k+1)} - 1) / (b - 1) = O(b^k)$$

Proof

Step 1. By definition, sum the nodes at every depth from 0 to k :

$$N(k) = b^0 + b^1 + b^2 + \dots + b^k$$

Step 2. Let S denote this sum, and multiply the entire equation by b :

$$\begin{aligned} S &= 1 + b + b^2 + \dots + b^k \\ bS &= b + b^2 + b^3 + \dots + b^{(k+1)} \end{aligned}$$

Step 3. Subtract the first equation from the second. Every interior term, b , b^2 , up to b^k , appears in both sums and cancels, leaving only the highest and the lowest term:

$$bS - S = b^{(k+1)} - 1$$

Step 4. Factor the left hand side and solve for S :

$$\begin{aligned} S(b - 1) &= b^{(k+1)} - 1 \\ S &= (b^{(k+1)} - 1) / (b - 1), \quad \text{valid since } b \neq 1 \end{aligned}$$

Step 5. Rewrite S to isolate the growth term. Split the fraction:

$$\begin{aligned} S &= (b^{(k+1)}) / (b - 1) - 1 / (b - 1) \\ S &= (b / (b - 1)) \cdot b^k - 1 / (b - 1) \end{aligned}$$

Step 6. Because b is a fixed constant greater than 1, the factor $b / (b - 1)$ and the term $1 / (b - 1)$ do not depend on k . As k grows, the term $(b / (b - 1)) \cdot b^k$ dominates the constant term, so by the definition of Big O:

$$N(k) = O(b^k)$$

This completes the proof of the lemma. In every task below, this bound is applied by substituting the correct meaning of k for that algorithm, namely the maximum depth m for DFS and GBFS, the shallowest goal depth d for BFS, or the effective cost bound depth C^/ϵ for UCS and A Star.*

1. Task 1 : Depth First Search (DFS)

1.1 Goal and Approach

- Implemented in the function `depthFirstSearch` inside `search.py`.
- Uses a LIFO Stack from `util.py` as the fringe or frontier.
- Maintains an explicit explored set so previously expanded states are never expanded again.
- Successors are generated in the mandatory order North then East then South then West.
- Returns a valid list of actions that leads Pacman from the start state to the goal state.

1.2 Algorithm Steps

- Push the start state onto the stack with an empty action path.
- Pop a state from the stack.
- If the popped state is the goal, return the accumulated action list.
- If the state has not been visited, mark it visited and push its unvisited successors, in North East South West order, with the extended path.
- Repeat until the stack is empty (failure) or a goal is popped.

1.3 Time Complexity Derivation

Let b be the branching factor and m be the maximum depth of the search tree. In the worst case, DFS must explore all the way down to depth m before either finding a goal or exhausting the tree.

Step 1. Apply the Lemma proved in Section 0 with $k = m$, since DFS may generate up to b nodes per expansion down to depth m :

$$N(m) = (b^{(m+1)} - 1) / (b - 1) = O(b^m)$$

Time Complexity: $O(b^m)$.

1.4 Space Complexity Derivation

Step 1. Unlike the time bound, DFS does not need to store the entire tree, only the single path currently being explored, together with the still unexpanded siblings branching off that path.

Step 2. At each of the m levels along the current path, expanding a node generates at most b children. Of these, 1 child continues to be explored deeper, while at most $(b - 1)$ sibling nodes remain stored on the stack, unexpanded, at that level.

Step 3. Summing the stored path nodes and the stored siblings across all m levels gives:

$$\text{Stored nodes} \leq m + m(b - 1) = m \cdot b$$

Step 4. Therefore:

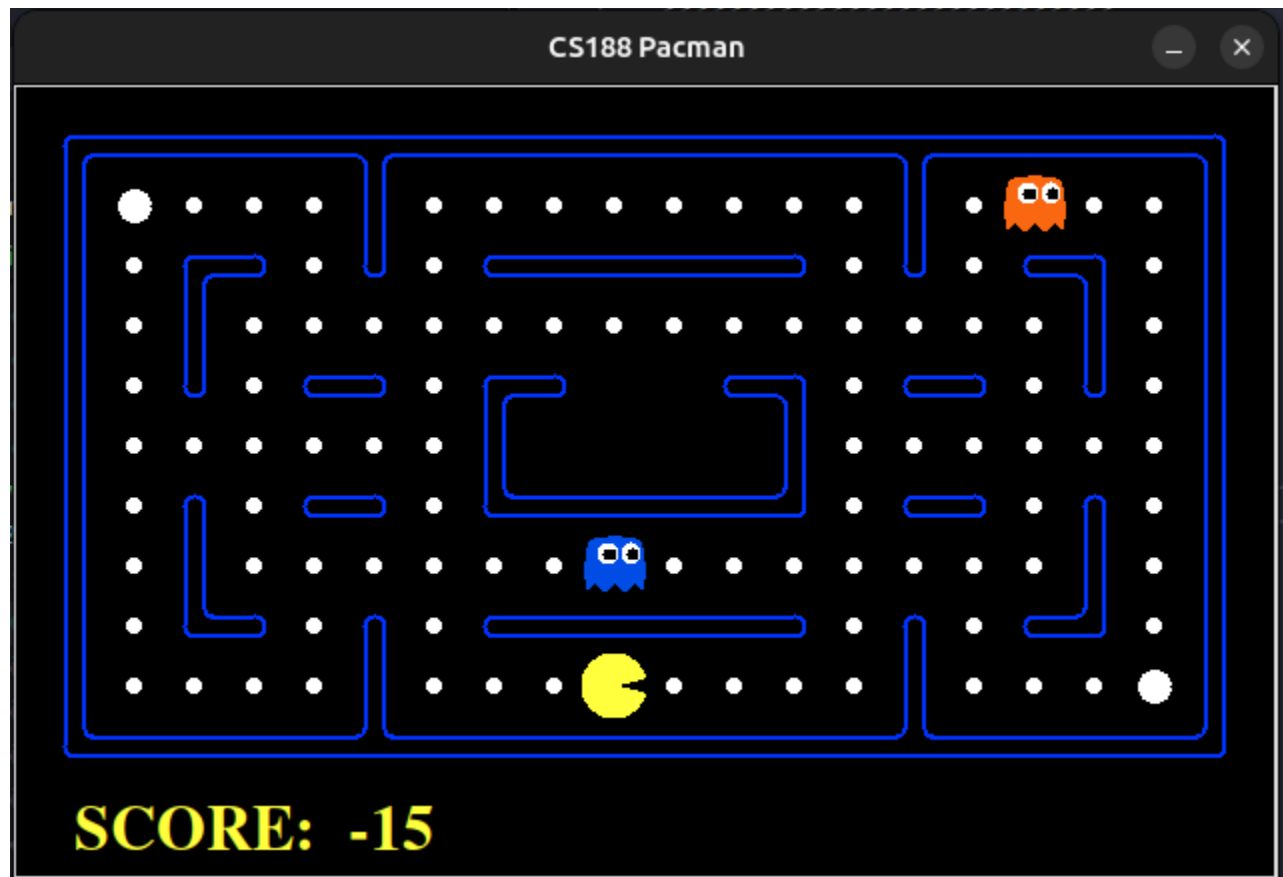
$$\text{Space Complexity} = O(b \cdot m)$$

This is the standard tree search bound. Because this implementation performs graph search using an explicit visited set, memory usage is also bounded above by $O(V)$, where V is the number of reachable states, so the true bound is $O(\min(bm, V))$.

1.5 Completeness and Optimality

- Complete: Yes, under the usual finite state and finite branching factor assumptions.
- Optimal: No. DFS can return a deeper or more expensive solution before a cheaper one is found.

1.6 Test Commands and Screenshots



2. Task 2 : Breadth First Search (BFS)

2.1 Goal and Approach

- Implemented in the function `breadthFirstSearch` inside `search.py`.
- Uses a FIFO Queue from `util.py` as the fringe or frontier.
- A state is marked visited as soon as it is enqueued, not when it is popped.
- This avoids re enqueueing a state that is already present in the frontier or explored set.
- Guarantees the shallowest, that is the optimal, path in terms of total action steps, when every step cost equals 1.

2.2 Time Complexity Derivation

Let b be the branching factor and d be the depth of the shallowest goal. BFS expands nodes level by level, so in the worst case (the goal is the very last node discovered at depth d), it must generate every node from depth 0 through depth d before terminating.

Step 1. Apply the Lemma from Section 0 with $k = d$:

$$N(d) = (b^{(d+1)} - 1) / (b - 1) = O(b^d)$$

Time Complexity: $O(b^d)$.

2.3 Space Complexity Derivation

Step 1. Because BFS expands level by level, the moment it is about to expand the nodes at depth d , its frontier (queue) contains exactly the set of all nodes generated at depth d , since every shallower node has already been expanded and every deeper node has not yet been generated.

Step 2. By the branching factor definition, the number of nodes at depth d is at most:

$$\text{Frontier size at depth } d \leq b^d$$

Step 3. Since the frontier must be held entirely in memory at this point:

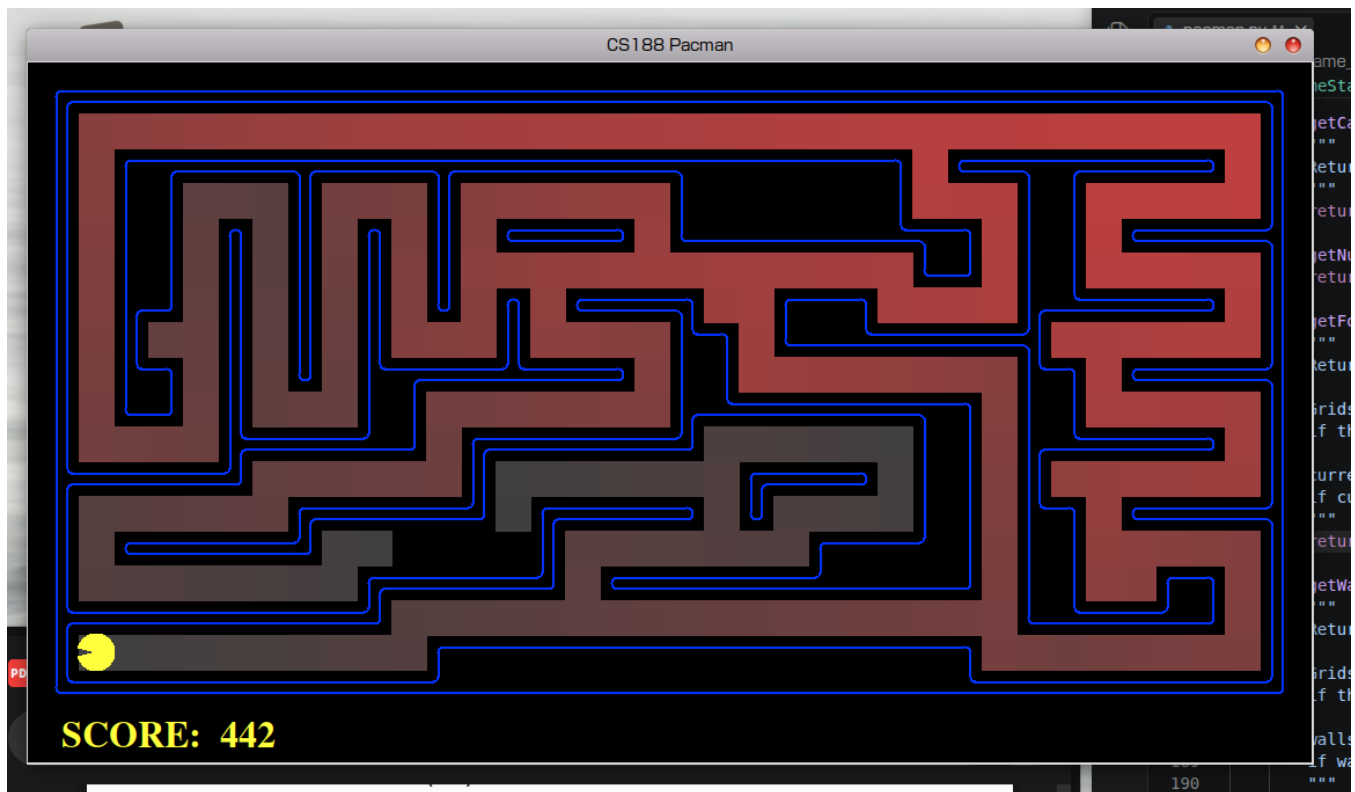
$$\text{Space Complexity} = O(b^d)$$

2.4 Completeness and Optimality

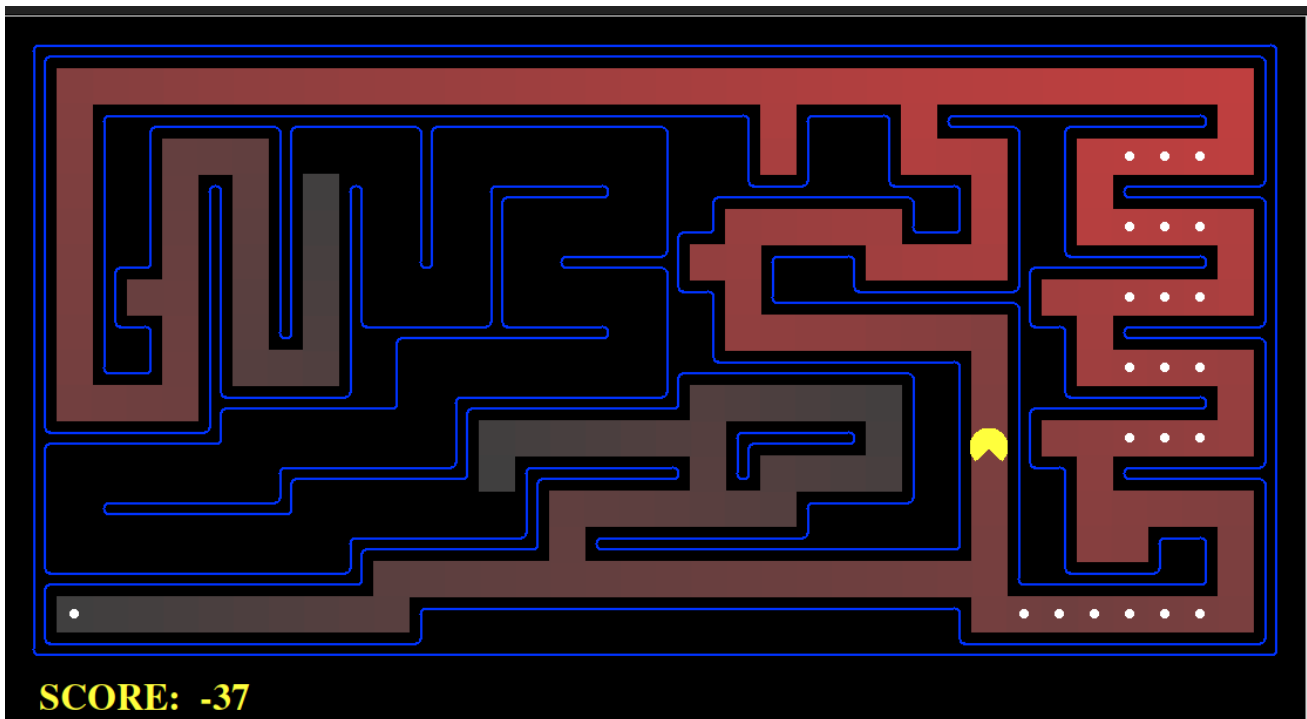
- Complete: Yes, provided the branching factor is finite.
- Optimal: Yes, when every action has the same, that is unit or constant, step cost.

2.5 Test Commands and Screenshots

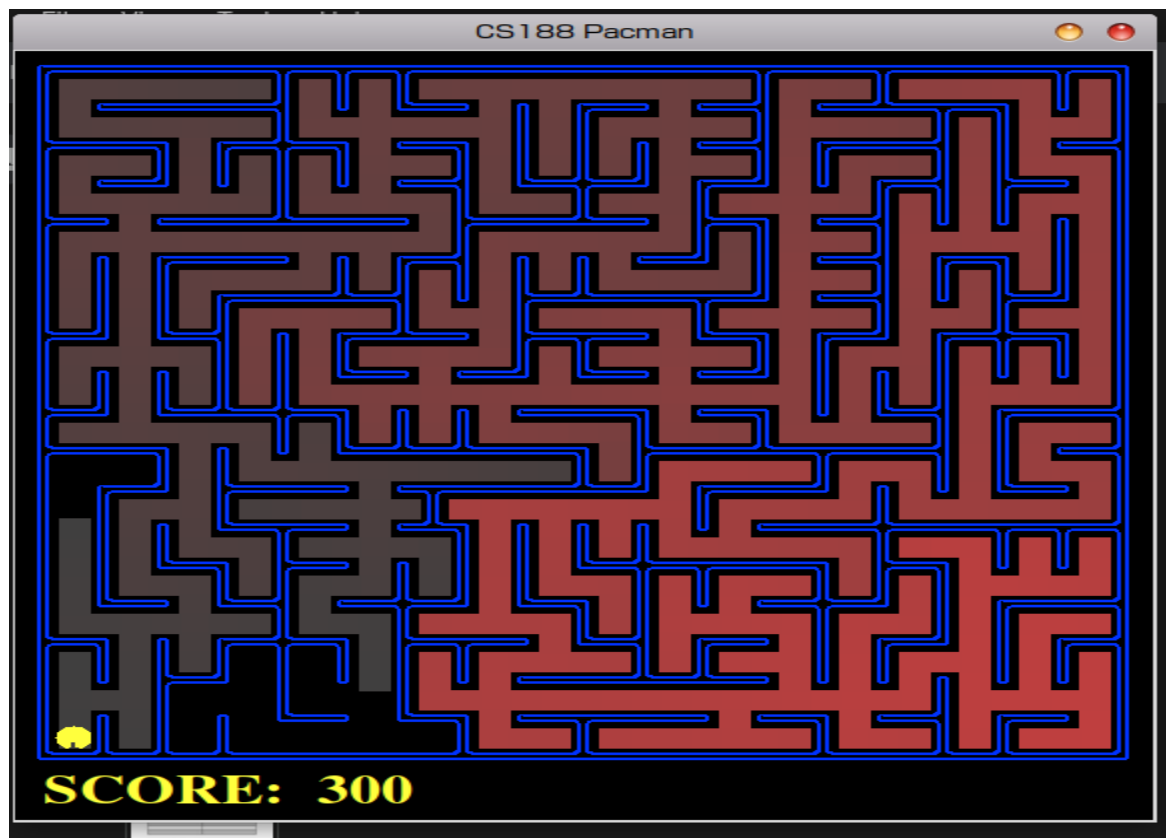
```
python pacman.py -l mediumMaze -p SearchAgent -a fn=bfs
```



```
python pacman.py -l mediumDenselyMaze -p SearchAgent -a fn=ucs
```



```
python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=bfs
```



3. Task 3 : Uniform Cost Search (UCS)

3.1 Goal and Approach

- Implemented in the function `uniformCostSearch` inside `search.py`.
- Uses a `PriorityQueue` from `util.py` ordered by accumulated path cost $g(n)$.
- A `bestCost` dictionary keeps track of the cheapest known cost to each state.
- If a newly generated successor offers a cheaper path cost to a state already on the fringe, its priority in the queue is updated.
- Tested across varying step costs, for example mazes with directional costs such as `stayEastSearch`.

3.2 Time Complexity Derivation

UCS measures progress by accumulated path cost, not by depth, so the derivation first converts a cost bound into an equivalent depth bound, then reuses the Lemma from Section 0.

Step 1. Let C^* be the cost of an optimal solution, and let ϵ ($\epsilon > 0$) be the smallest possible step cost in the state space.

Step 2. Consider any node n reached after exactly k steps, with accumulated path cost $g(n)$. Since every step costs at least ϵ :

$$g(n) \geq k \cdot \epsilon$$

Step 3. UCS only needs to expand nodes whose path cost does not exceed the optimal cost, that is $g(n) \leq C^*$. Combining this with Step 2:

$$k \cdot \epsilon \leq g(n) \leq C^*$$

Step 4. Divide both sides by ϵ to isolate k :

$$k \leq C^* / \epsilon$$

Step 5. So every node that UCS might expand lies at an effective depth of at most $L = C^* / \epsilon$. Applying the Lemma from Section 0 with $k = L$:

$$N(L) = O(b^L) = O(b^{C^*/\epsilon})$$

Time Complexity: $O(b^{C^*/\epsilon})$.

Implementation note. Since UCS uses a heap based priority queue with up to $O(V)$ entries, each heap insertion or removal costs about $O(\log V)$, giving an implementation level bound of roughly $O((V + E) \log V)$, where V and E are the number of reachable states and edges respectively.

3.3 Space Complexity Derivation

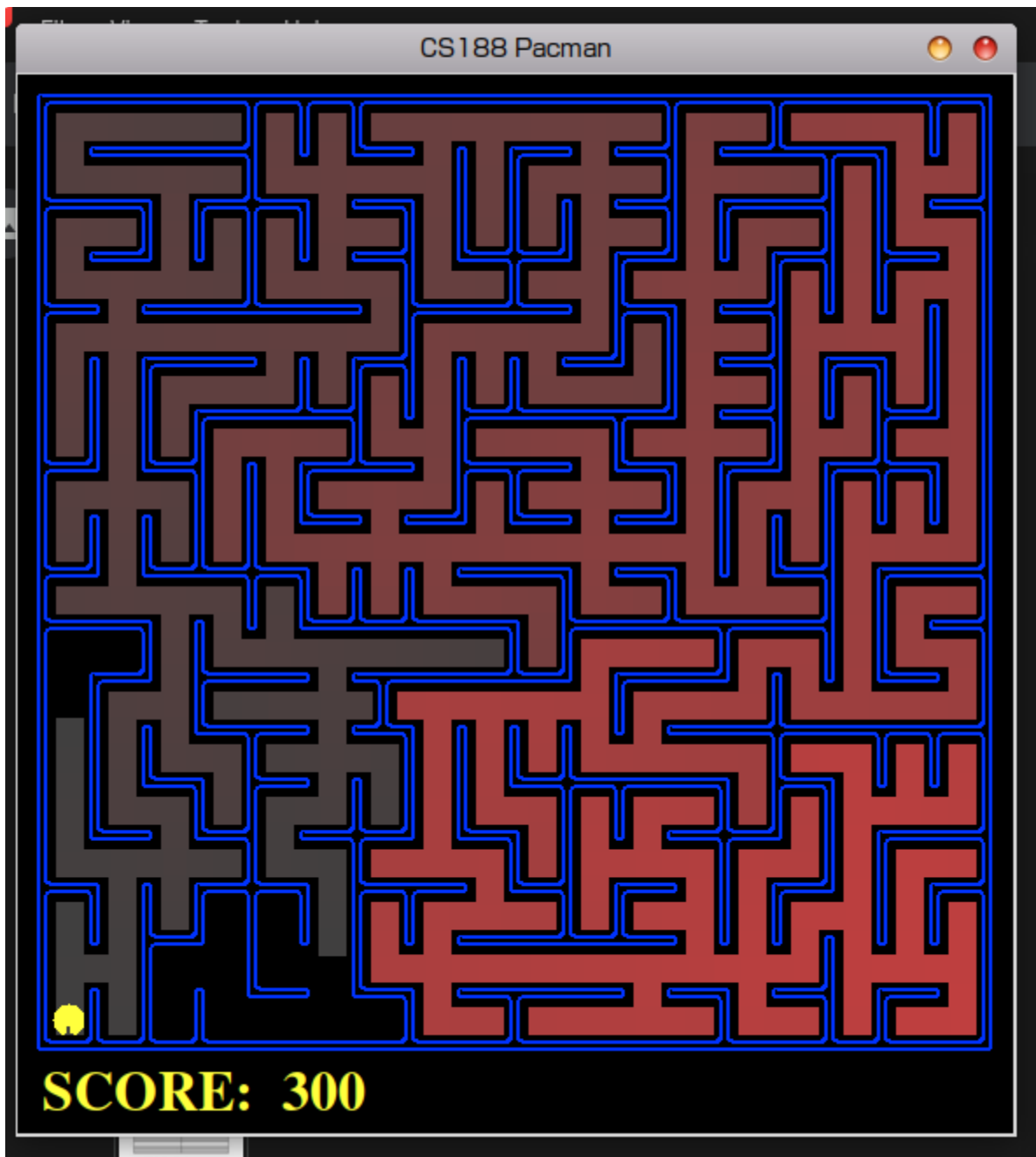
Step 1. By the same effective depth argument as Step 5 above, the frontier can, in the worst case, contain all nodes up to effective depth $L = C^* / \epsilon$.

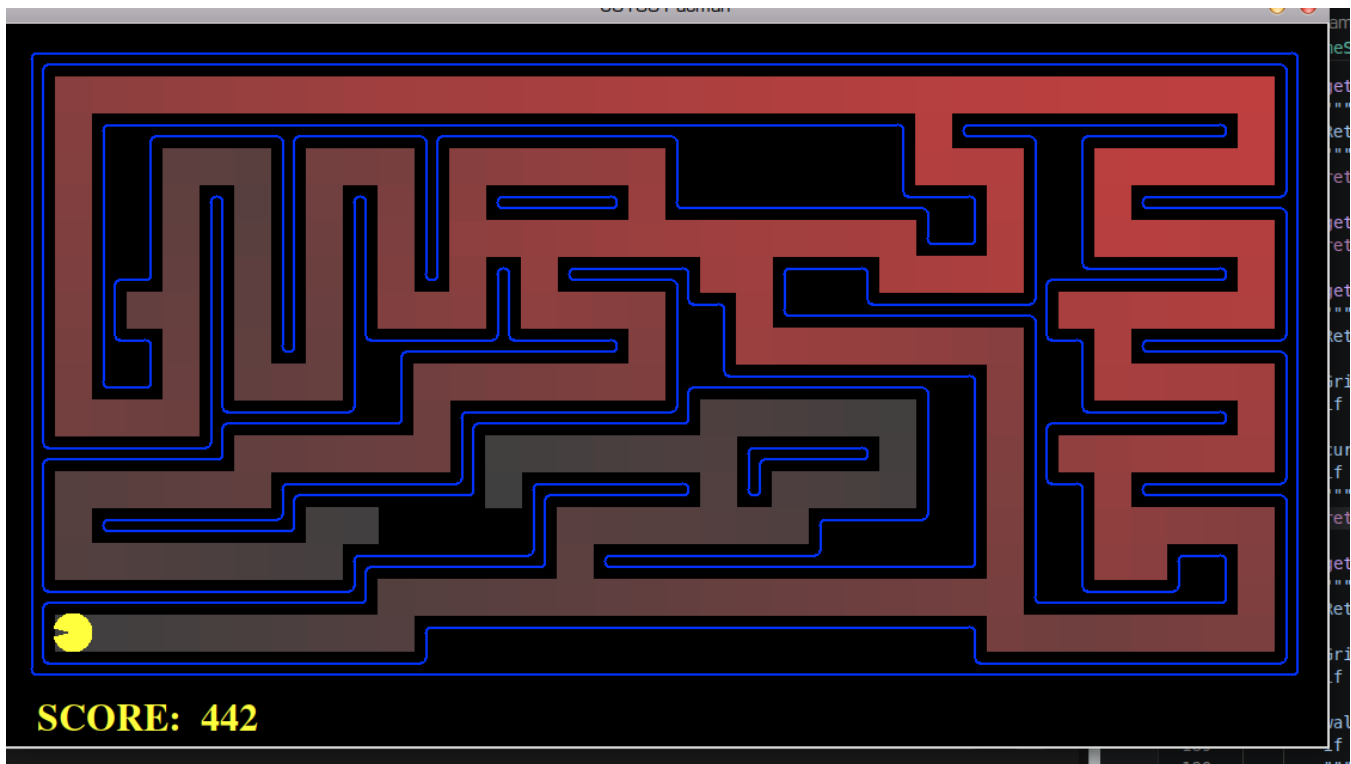
$$\text{Space Complexity} = O(b^{C^*/\epsilon})$$

3.4 Completeness and Optimality

- Complete: Yes, if every step cost is at least some positive ϵ .
- Optimal: Yes, under the usual assumption that step costs are positive.

3.5 Test Commands and Screenshots





4. Task 4 : Greedy Best First Search (GBFS)

4.1 Goal and Approach

- Implemented in the function `greedyBestFirstSearch`, aliased as `gbfs`, inside `search.py`.
- Uses a `PriorityQueue` ordered strictly by heuristic value $h(n)$.
- The accumulated cost $g(n)$ already travelled is completely ignored when ordering the queue.
- The heuristic is passed dynamically through the command line, for example `heuristic=manhattanHeuristic`, so different heuristics can be swapped without changing `search.py`.

4.2 Time Complexity Derivation

Greedy search orders the frontier only by $f(n) = h(n)$, with no term that accounts for cost already spent. This means an inaccurate heuristic gives the algorithm no mechanism to avoid exploring a long, unproductive branch.

Step 1. In the worst case, the heuristic can consistently rank a state on a long, unproductive branch as more promising than a state that leads directly to the goal, causing GBFS to expand nodes down to the full depth m of the tree before backtracking, exactly as in the DFS worst case.

Step 2. Since up to b nodes can be generated per expansion down to depth m , apply the Lemma from Section 0 with $k = m$:

$$N(m) = (b^{(m+1)} - 1) / (b - 1) = O(b^m)$$

Time Complexity: $O(b^m)$, worst case.

4.3 Space Complexity Derivation

Step 1. Because a misleading heuristic can cause GBFS to keep large parts of a bad branch, together with unexplored siblings, on the frontier at once, the same worst case node count from Step 2 above bounds the number of nodes that may need to be stored:

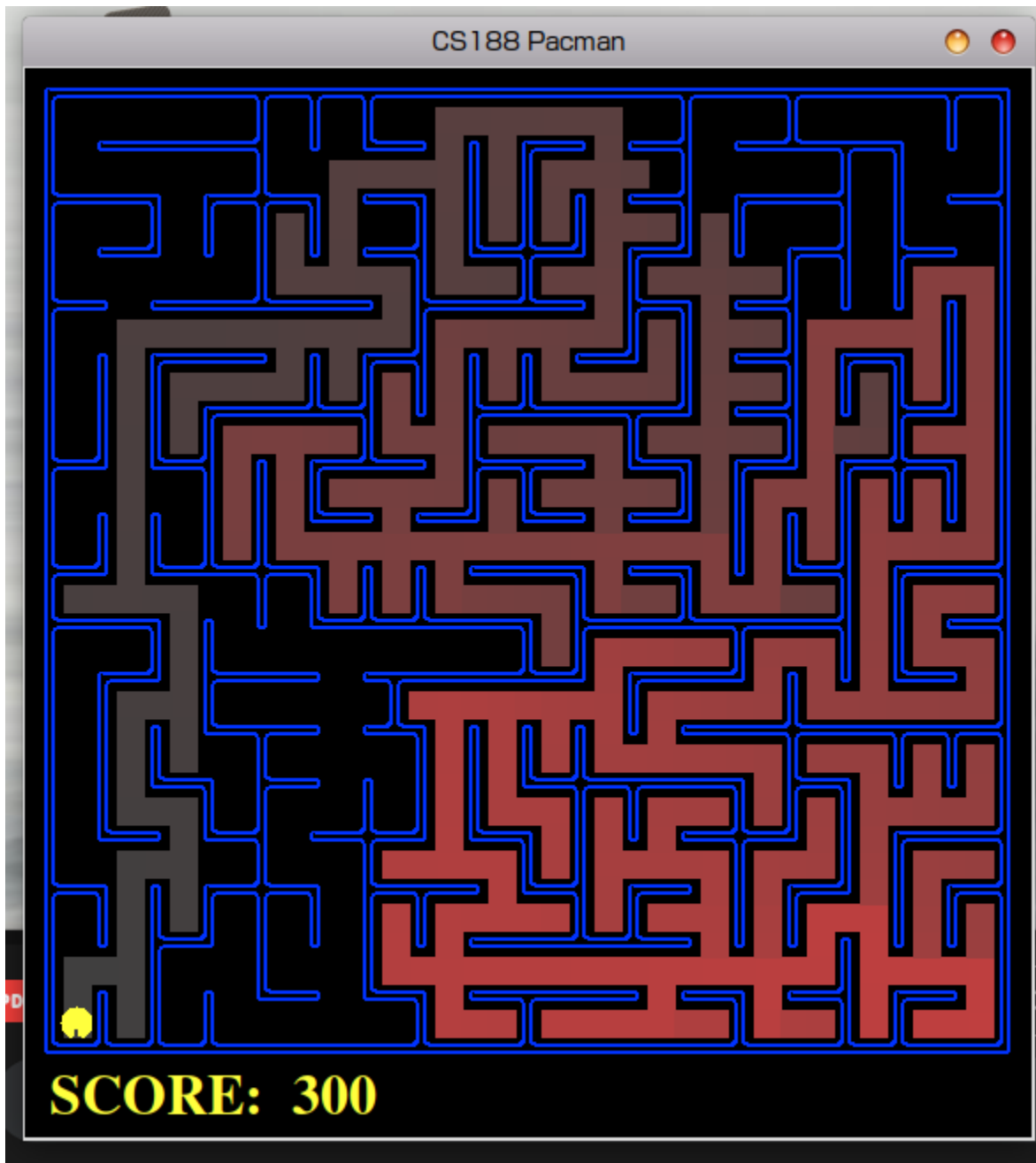
Space Complexity = $O(b^m)$, worst case

For this graph search implementation, the explicit visited set gives an additional bound of $O(V)$, where V is the number of reachable states.

4.4 Completeness and Optimality

- Complete: Under finite state and finite branching factor assumptions, with the usual caveats.
- Optimal: No. Greedy search only considers $h(n)$, so a misleading heuristic can cause an expensive solution to be chosen instead of a cheaper one.

4.5 Test Commands and Screenshots



5. Task 5 : A Star Search

5.1 Goal and Approach

- Implemented in the function `aStarSearch` inside `search.py`.
- Uses a `PriorityQueue` ordered by the evaluation function $f(n) = g(n) + h(n)$.
- $g(n)$ is the accumulated path cost from the start state.
- $h(n)$ is the estimated cost from n to the nearest goal.
- Passing `heuristic=nullHeuristic` makes A Star behave exactly like Uniform Cost Search, which we used as a correctness check.

5.2 Admissibility and Consistency, Defined

Let $h^*(n)$ denote the true cheapest cost from state n to a goal.

- A heuristic h is admissible when $h(n) \leq h^*(n)$ for every state n , that is, it never overestimates the true remaining cost.
- A heuristic h is consistent when $h(n) \leq c(n, n') + h(n')$, for every edge from n to a successor n' with step cost $c(n, n')$.
- `nullHeuristic` always returns 0, so $h(n) = 0 \leq h^*(n)$ always holds, and it is trivially admissible and consistent.
- `manhattanHeuristic` is admissible and consistent on grid mazes, since diagonal movement is not permitted, so the straight line Manhattan distance can never exceed the true number of grid steps required.

5.3 Time and Space Complexity Derivation

Step 1. Case $h(n) = 0$ for all n (`nullHeuristic`). Then $f(n) = g(n) + 0 = g(n)$, which is exactly the priority rule used by UCS. Every step of the UCS derivation in Section 3.2 therefore applies unchanged:

$$\text{Time Complexity} = O(b^{(C^*/\epsilon)})$$

Step 2. General case with an admissible heuristic h . Since $h(n) \geq 0$ for the heuristics used in this project, $g(n) \leq f(n)$. A Star only needs to expand nodes with $f(n) \leq C^*$ (nodes with $f(n) > C^*$ cannot improve on a solution of cost C^* and need not be expanded). So for any expanded node reached in k steps:

$$k \cdot \epsilon \leq g(n) \leq f(n) \leq C^*$$

Step 3. Dividing by ϵ gives the same bound on k as in the UCS derivation:

$$k \leq C^* / \epsilon$$

Step 4. Applying the Lemma from Section 0 with $k = L = C^*/\epsilon$:

$$N(L) = O(b^{(C^*/\epsilon)})$$

Time Complexity: $O(b^{(C^*/\epsilon)})$, worst case. **Space Complexity:** $O(b^{(C^*/\epsilon)})$, worst case, by the same frontier storage argument used for UCS.

5.4 Formal Proof That A Star Is Optimal With An Admissible Heuristic

Theorem. If h is admissible, then A Star (tree search) returns a solution of optimal cost C^* .

Proof by contradiction.

Step 1. Let $P^* = (n_0 = \text{start}, n_1, n_2, \dots, n_g = \text{goal}^*)$ be an optimal solution path, with total cost C^* .

Step 2. Suppose, for contradiction, that A Star instead selects and returns a different, suboptimal goal G_2 , with:

$$g(G_2) = C_2 \quad \text{and} \quad C_2 > C^*$$

Step 3. Because the optimal goal has not yet been selected at the moment G_2 is chosen, there must exist some node n lying on the optimal path P^* that is still present on the frontier. This holds because to expand any node on P^* , its parent must first have been expanded, so if the true goal has not yet been reached, at least one ancestor along P^* remains on the frontier, waiting to be expanded.

Step 4. Since h is admissible, $h(n) \leq h^*(n)$ for this frontier node n . Also, since n lies on the optimal path, the cost already spent to reach n plus the true remaining cost to the goal equals exactly C^* :

$$g(n) + h^*(n) = C^*$$

Step 5. Combine the admissibility inequality with Step 4:

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*$$

Step 6. Now evaluate f at the suboptimal goal G_2 . Since G_2 is itself a goal state, its heuristic value is 0, so:

$$f(G_2) = g(G_2) + h(G_2) = g(G_2) + 0 = C_2$$

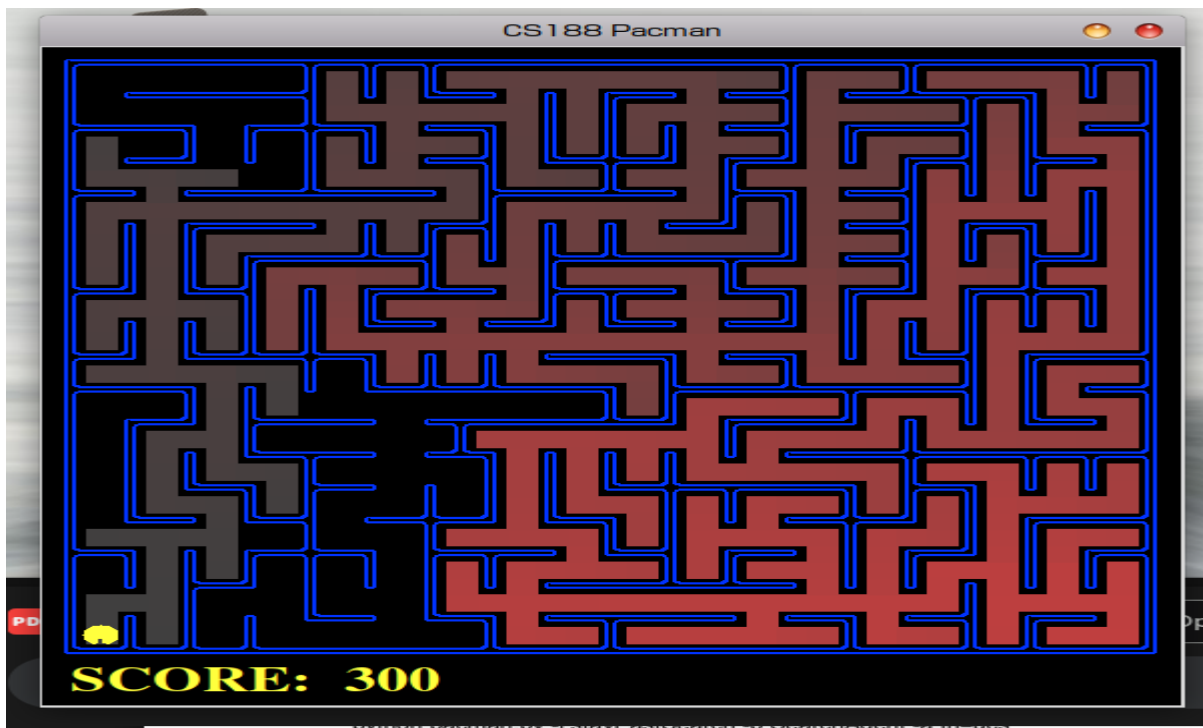
Step 7. Combining Step 5, Step 6 and the assumption $C_2 > C^*$ from Step 2:

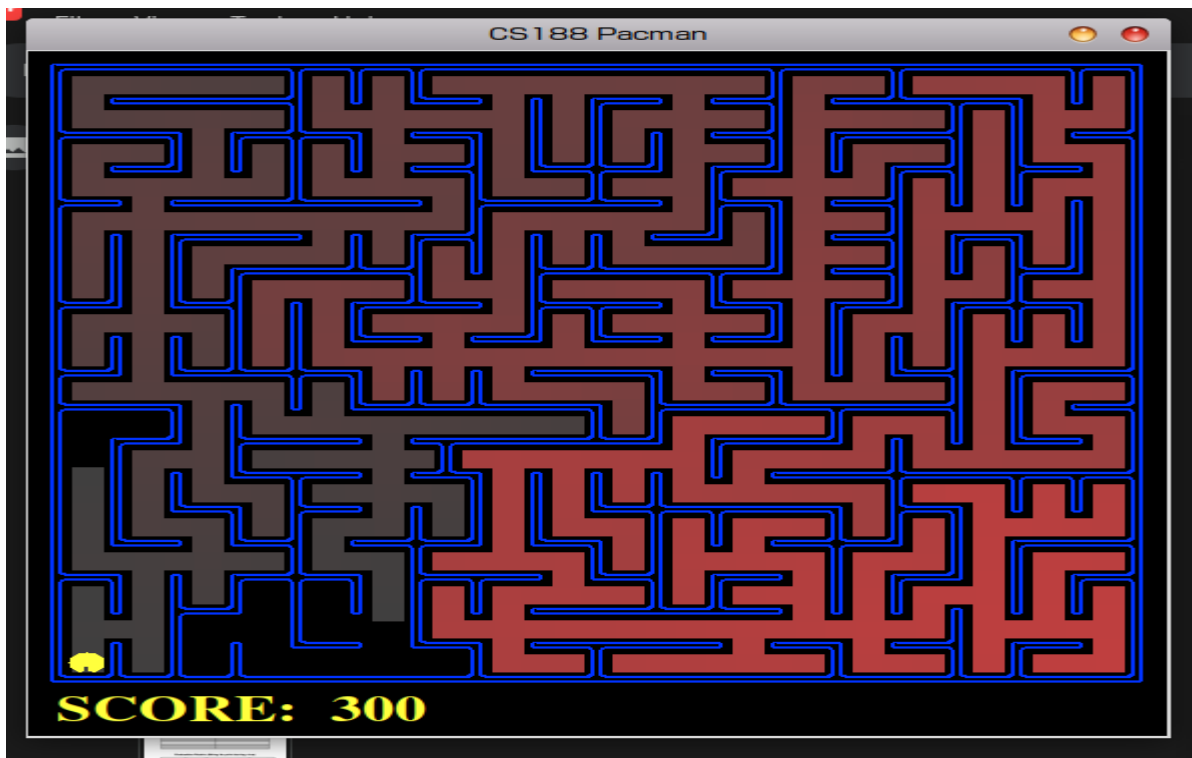
$$f(n) \leq C^* < C_2 = f(G_2)$$

Step 8. A Star always selects the frontier node with the lowest f value for expansion next. Since $f(n) < f(G_2)$, node n must be selected and expanded before G_2 can be selected. This directly contradicts the assumption in Step 2 that G_2 was the node chosen and returned.

Step 9. Since the assumption in Step 2 leads to a contradiction, no suboptimal goal can be returned before the true optimal goal. Therefore A Star must return a solution of cost exactly C^* .

5.5 Test Commands and Screenshots





6. Task

6 : Multi Goal State Space (Corners Problem)

6.1 Goal and Approach

- Formulated in searchAgents.py to require Pacman to visit all four corners of a maze in the minimum number of steps.
- State representation, CornersProblem: a tuple containing (pacman_position, tuple_of_visited_corners).
- tuple_of_visited_corners records which of the four corners have already been reached.
- This keeps the state space finite and hashable while still capturing exactly the information needed for the goal test.
- Goal test succeeds only when all four corners have been visited.

6.2 cornersHeuristic Design and Admissibility Proof

Let U be the set of corners not yet visited in a given state, and let pos be Pacman's current position. Define the heuristic as the length of a short walking order through the remaining corners in U , computed using Manhattan distance between consecutive points, for example by always stepping next to the nearest remaining unvisited corner.

Step 1. Let $h^*(n)$ denote the true cost of a shortest path, inside the actual maze, that starts at pos and visits every corner in U at least once.

Step 2. Any actual path through the maze between two points p and q must take at least as many steps as the Manhattan distance between p and q , because each maze move changes exactly one coordinate by 1, and walls can only lengthen a route, never shorten it below the straight line grid distance:

$$\text{maze_distance}(p, q) \geq \text{manhattan_distance}(p, q)$$

Step 3. The true optimal tour $h^*(n)$ is a sum of true maze distances between consecutive points on some order of visiting U . Replacing each true maze distance with its Manhattan distance lower bound, from Step 2, can only decrease or keep equal the total:

$$h(n) = \text{sum of Manhattan distances along a chosen order} \leq \text{sum of true maze distances along that order}$$

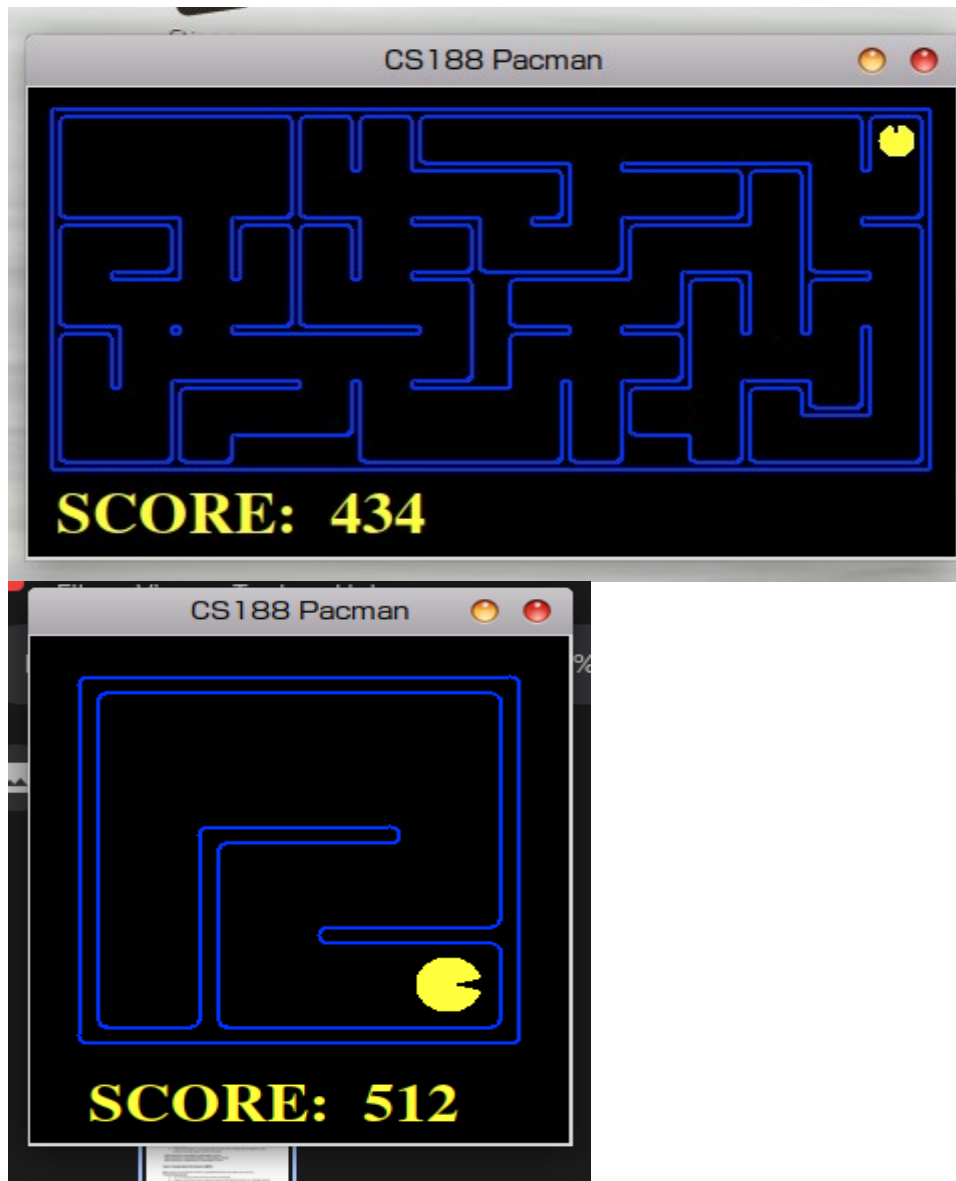
Step 4. Since the true optimal cost $h^*(n)$ is itself a sum of true maze distances over some (possibly different, but at least as good) order:

$$h(n) \leq h^*(n)$$

Therefore cornersHeuristic is admissible.

- Consistency follows in the same style: moving one true maze step toward the nearest unvisited corner can reduce the remaining Manhattan distance estimate by at most 1, which equals the cost of that one step, satisfying $h(n) \leq c(n, n') + h(n')$ for every edge.

6.3 Test Commands and Screenshots



7. Task 7 : Eating All Food Dots and Nearest Food Search

7.1 foodHeuristic for FoodSearchProblem, and Its Admissibility

Let F be the set of remaining food positions in a state, and let pos be Pacman's current position. Define $foodHeuristic(n)$ as the weight of a minimum spanning tree built over the point set $F \cup \{pos\}$, using Manhattan (or maze) distance as edge weight between every pair of points.

Step 1. Let $h^*(n)$ be the true cost of the shortest actual path that starts at pos and visits every point in F .

Step 2. Any path that visits every point in F , starting from pos , defines a connected sub graph (in fact a walk, hence also a connected structure) spanning every point in $F \cup \{pos\}$. The total length of that path is therefore at least as large as the weight of the minimum spanning tree over the same point set, since a minimum spanning tree is, by definition, the lowest total edge weight needed to connect a set of points:

$$h^*(n) \geq \text{weight of Minimum Spanning Tree over } F \cup \{pos\} = \text{foodHeuristic}(n)$$

Step 3. Rearranging Step 2 directly gives the admissibility condition:

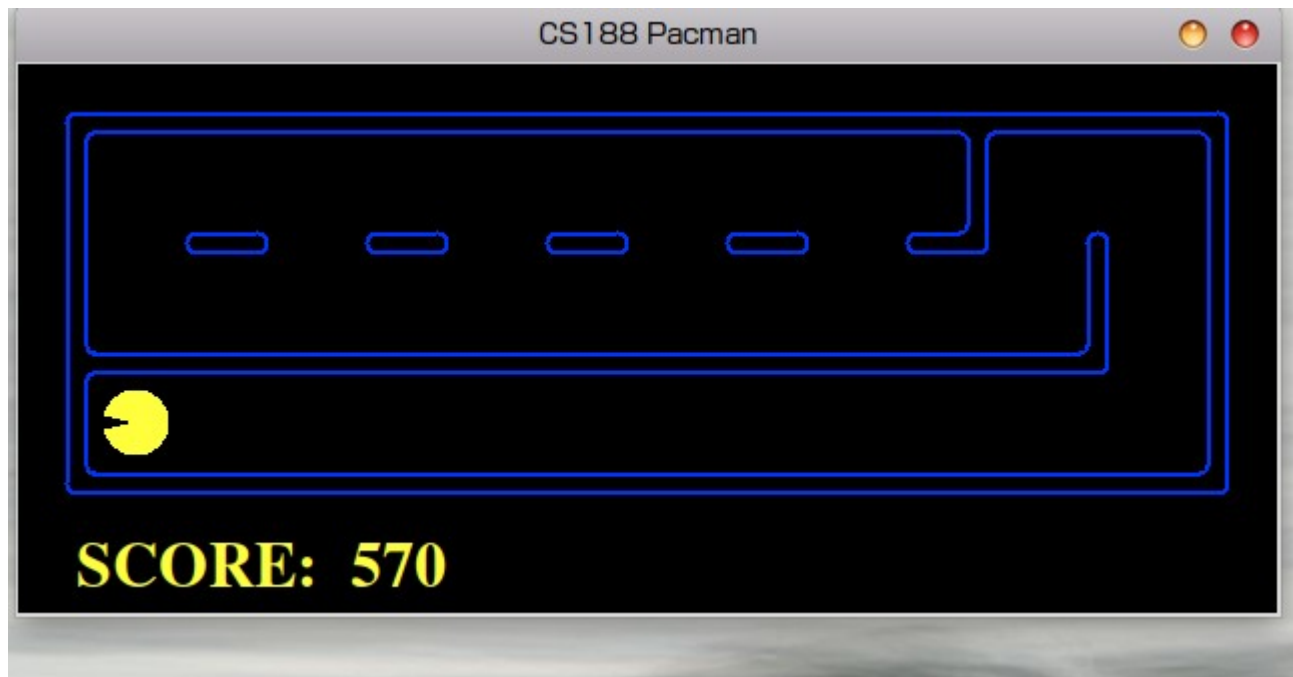
$$\text{foodHeuristic}(n) \leq h^*(n)$$

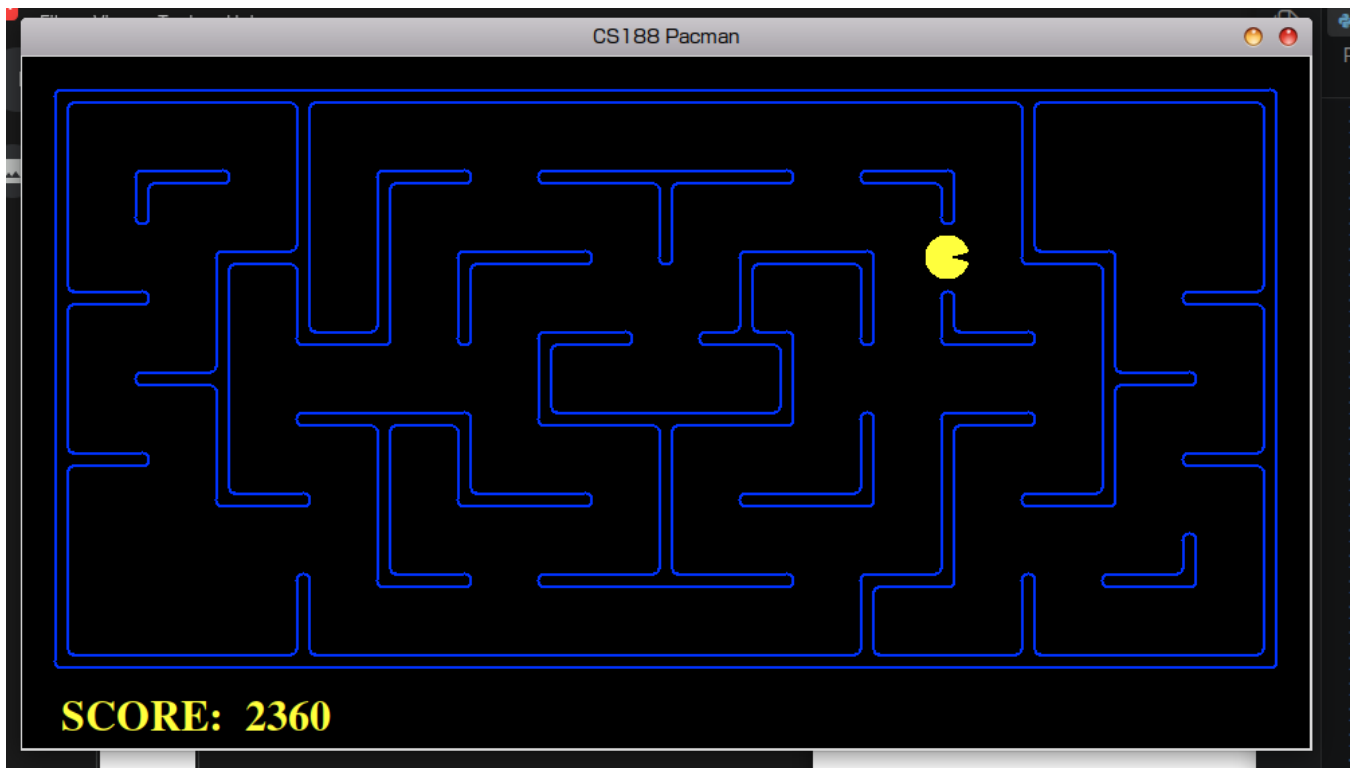
Therefore $foodHeuristic$ is admissible. It is also considerably tighter than a trivial heuristic such as simply counting the remaining food dots, because it accounts for the actual spatial layout of the remaining dots.

7.2 AnyFoodSearchProblem

- Redefines the goal test to succeed as soon as Pacman reaches any single food dot, rather than all of them.
- Solved using `breadthFirstSearch`, which guarantees finding the closest food dot in terms of action step count, since BFS is optimal for unit step costs (proved in Section 2.4).

7.3 Test Commands and Screenshots





8. Automated CSV Trace Logging

- Every algorithm execution writes a state by state trace to a CSV file inside the evidence folder.
- Each row corresponds to one node expansion.

Mandatory CSV columns:

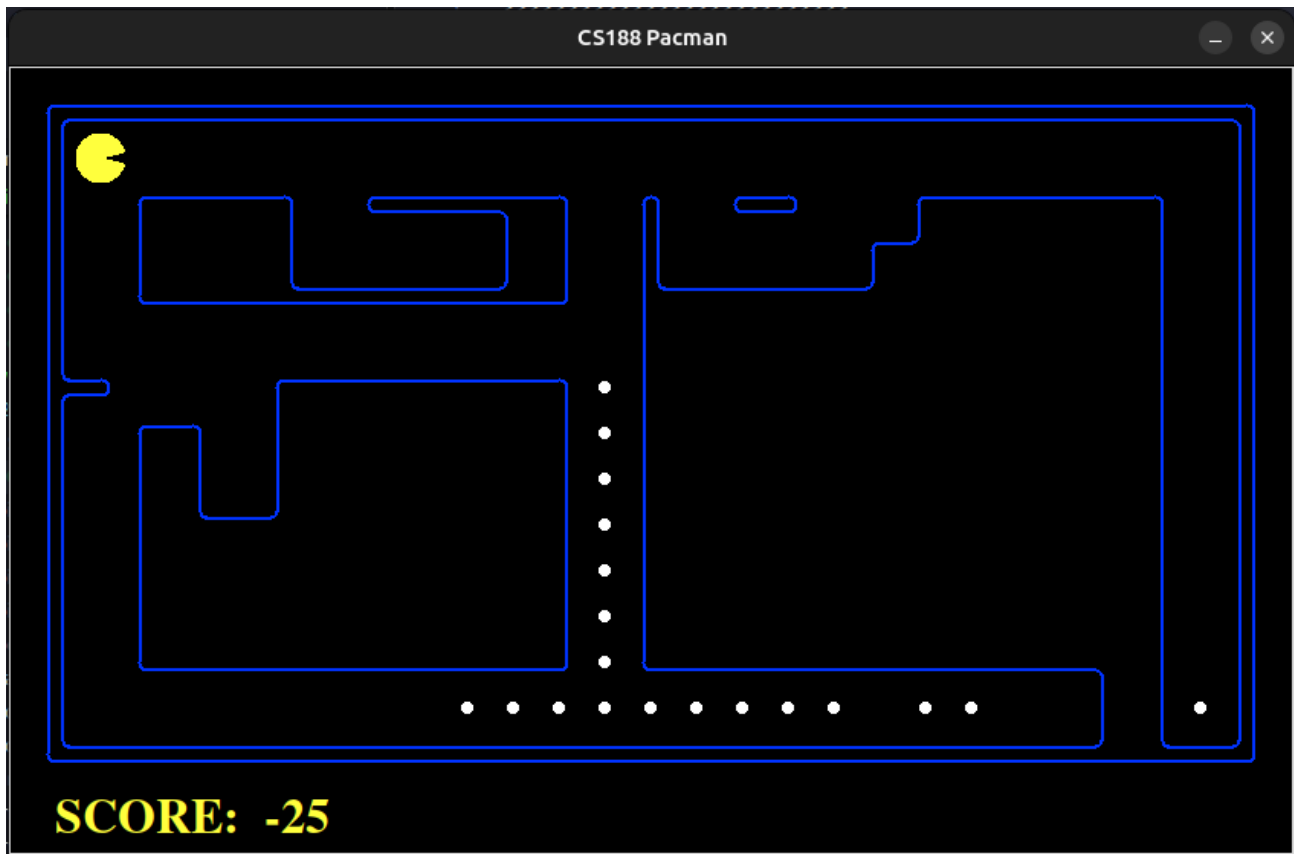
Column	Description
iteration	Sequential counter for each node expansion.
expanded_state	The state popped from the fringe and expanded on this iteration.
parent	The state from which the expanded state was generated.
action	The action taken to reach the expanded state from its parent.
generated_successors	Number or list of successor states generated during this expansion.
frontier_before	Size or contents of the fringe before this expansion.
frontier_after	Size or contents of the fringe after successors are added.
explored	Size of the explored or visited set at this point.
g	Accumulated path cost $g(n)$ at this state, used by UCS and A Star.
h	Heuristic value $h(n)$ at this state, used by GBFS and A Star.
f	Evaluation function $f(n) = g(n) + h(n)$ for A Star, or $f(n) = h(n)$ for GBFS.

- The logger appends one row per expansion during each of the five search functions.
- A new, uniquely named CSV file is written for every run, for example based on layout name and algorithm, so logs from different tasks and mazes do not overwrite each other inside evidence.

9. Custom Maze Creation and Experiments

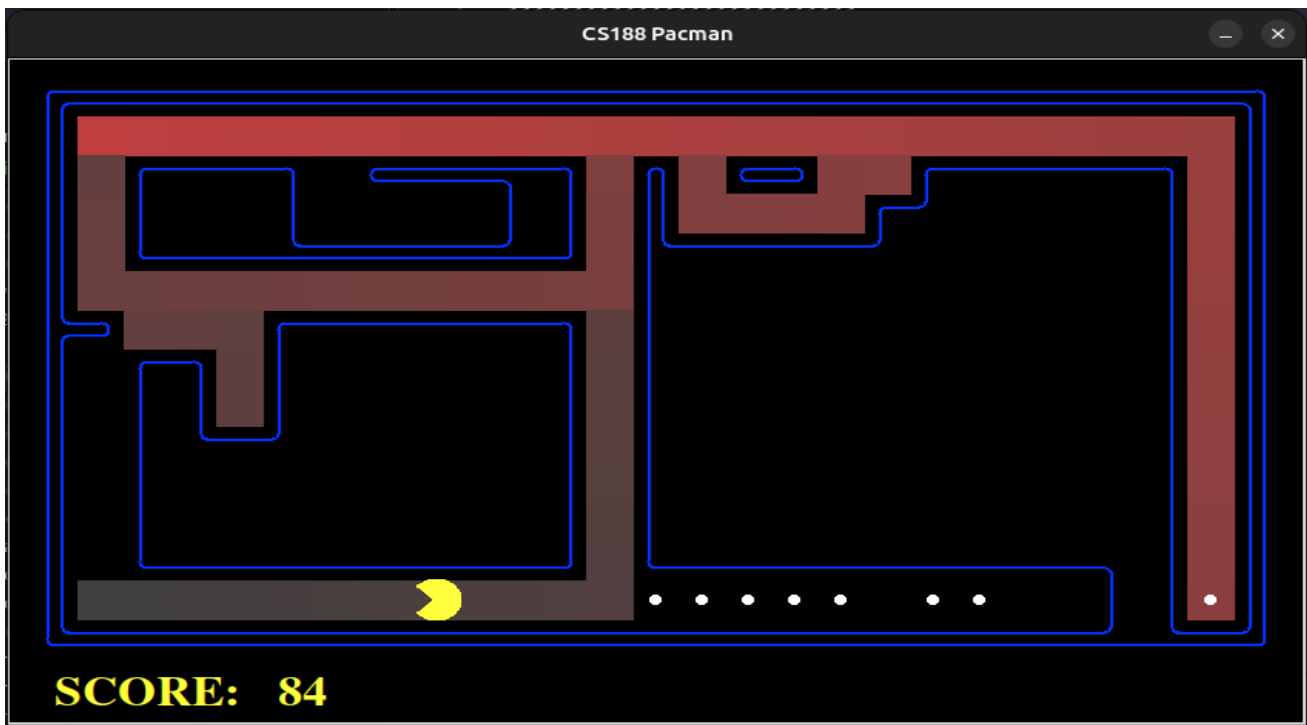
9.1 Layout Design

- Contains multiple decision branches, dead ends, and deceptive traps.
- Designed so that Greedy Best First Search commits early to a heuristically attractive but longer path, while A Star correctly avoids the trap by also accounting for accumulated cost $g(n)$.

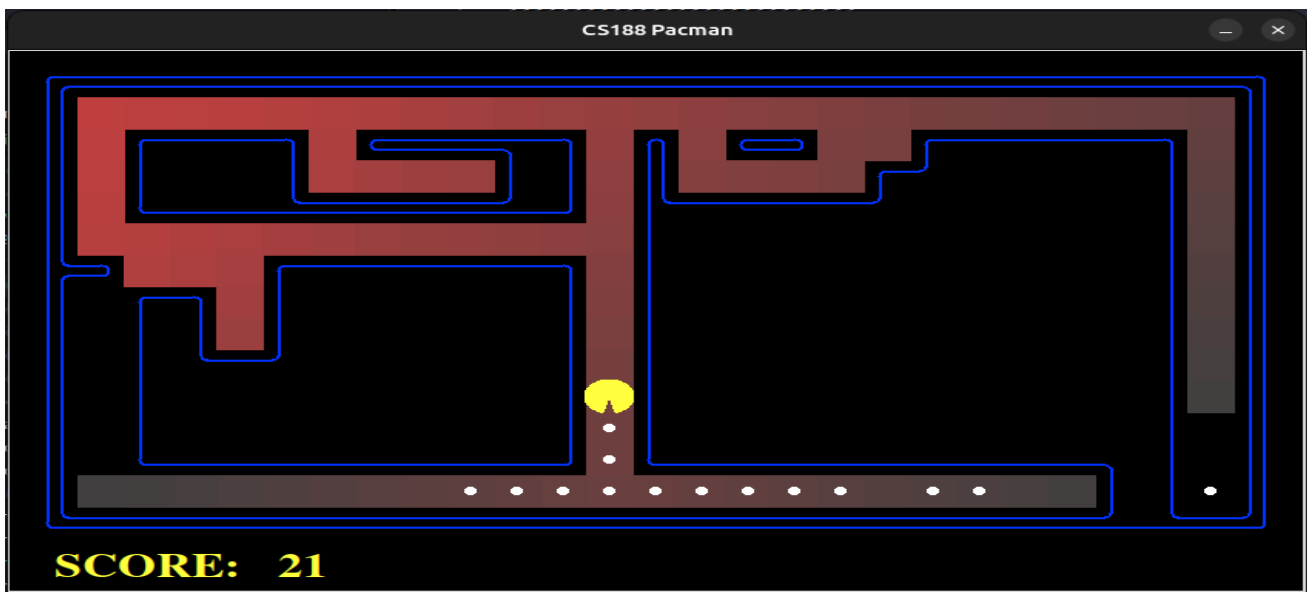


9.2 Running All Five Algorithms on the Custom Layout

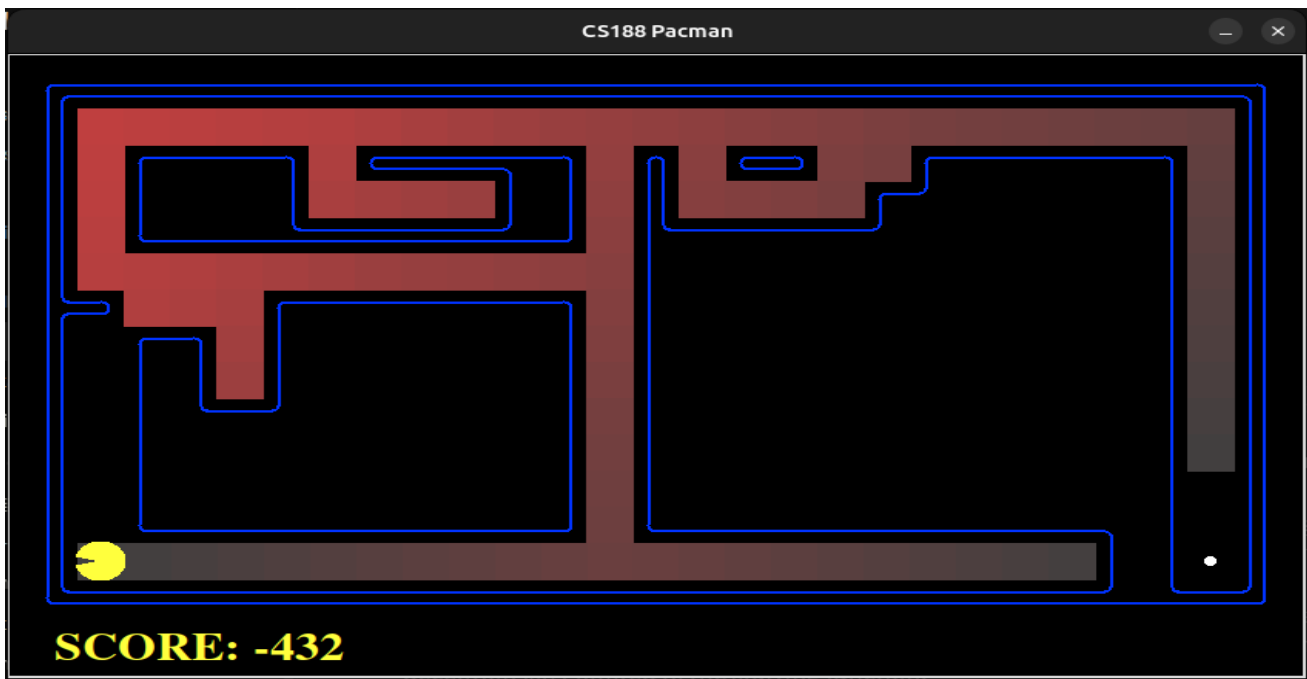
```
python pacman.py -l 24i3082Search -p SearchAgent -a fn=dfs
```



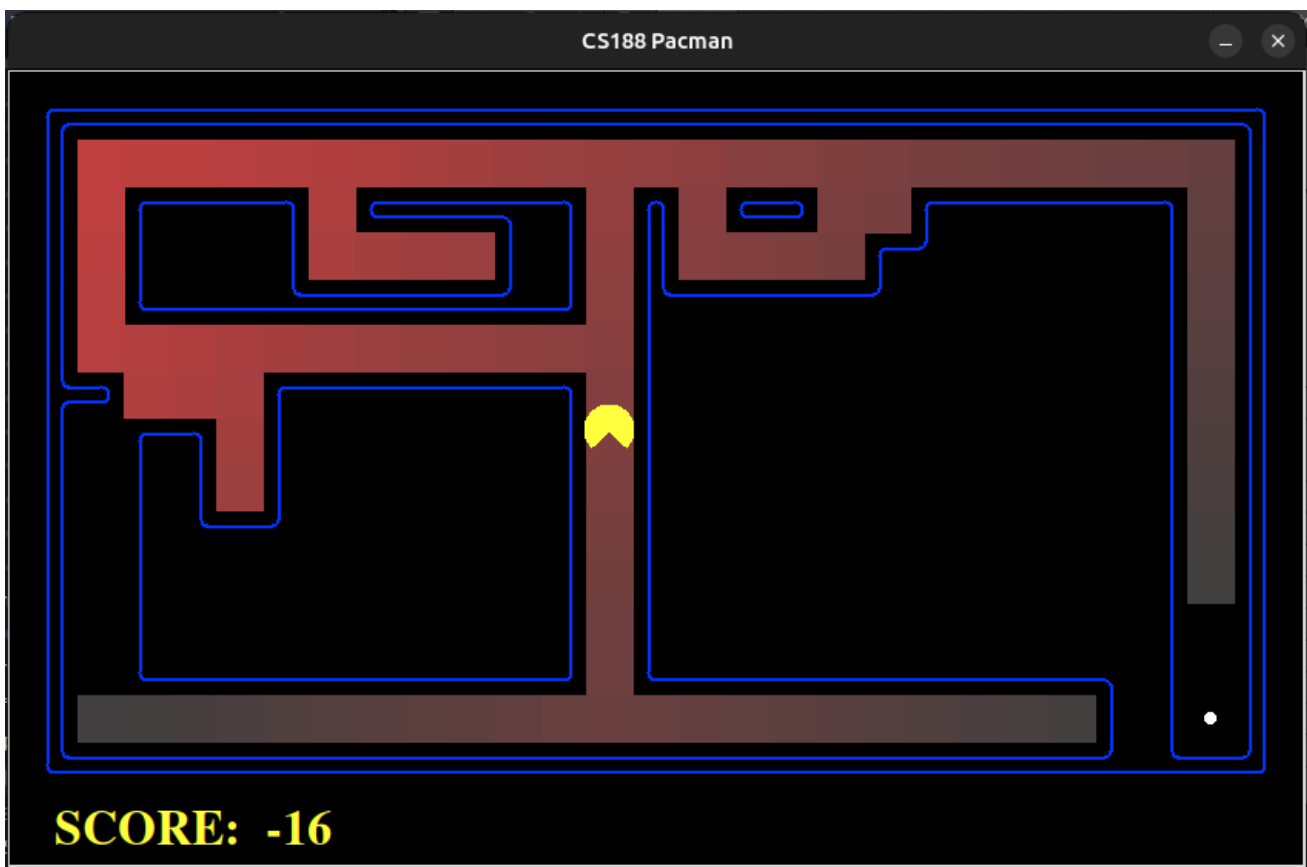
```
python pacman.py -l 24i3082Search -p SearchAgent -a fn=bfs
```



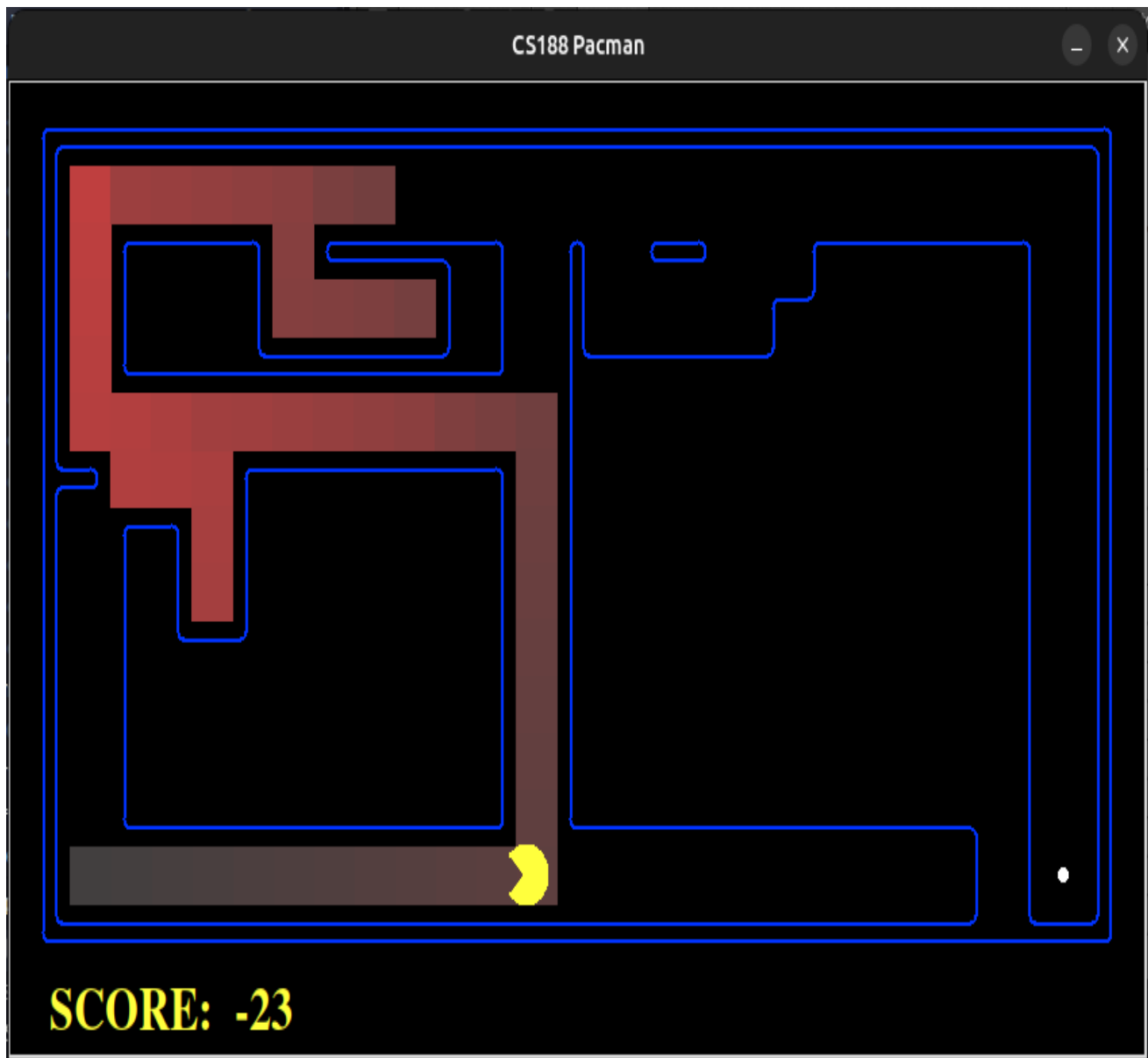
```
python pacman.py -l 24i3082Search -p SearchAgent -a fn=ucs
```



```
python pacman.py -l 24i3082Search -p SearchAgent -a  
fn=gbfs,heuristic=manhattanHeuristic
```



```
python pacman.py -l 24i3082Search -p SearchAgent -a  
fn=astar,heuristic=manhattanHeuristic
```



9.3 Empirical Comparison

Fill in the table below using the values recorded in the evidence CSV logs for the custom layout runs.

Algorithm	Path Cost	Nodes Expanded	Optimal (Yes/No)
DFS	36	37	No*
BFS	36	98	Yes
UCS	36	98	Yes
GBFS	40	95	No
A*	36	98	Yes

10. Complexity Summary Table

Algorithm	Time Complexity	Space Complexity	Complete	Optimal
DFS	$O(b^m)$	$O(bm)$, or $O(V)$ for this graph search code	Yes	No
BFS	$O(b^d)$	$O(b^d)$	Yes	Yes, for unit step cost
UCS	$O(b^{(C^*/\epsilon)})$	$O(b^{(C^*/\epsilon)})$	Yes, if step cost is positive	Yes
GBFS	$O(b^m)$, worst case	$O(b^m)$, or $O(V)$ for this graph search code	Yes, with the usual caveats	No
A Star	$O(b^{(C^*/\epsilon)})$, worst case	$O(b^{(C^*/\epsilon)})$, worst case	Yes, if step cost is positive	Yes, with an admissible heuristic

Full derivations for every row above are given in the Complexity Derivation subsections of Task 1 through Task 5, all built from the single Lemma proved in Section 0.